SOFTWARE ENGINEERING STUDY ASSISTANT

A Local Retrieval-Augmented Generation Application Using Foundry Local

Prepared by: Suheyla Ceren Mescioglu

Abstract

This project presents a private and fully local Software Engineering Study Assistant developed using Retrieval-Augmented Generation, commonly known as RAG. The purpose of the application is to allow students to ask questions about Software Engineering concepts and receive concise answers grounded in a local collection of study documents.

The application uses Foundry Local and the Phi-3.5 Mini language model for on-device artificial intelligence inference. Node.js and Express are used for the back-end server, SQLite is used for document storage, and TF-IDF with cosine similarity is used to retrieve relevant document chunks. The front end was developed using HTML, CSS, and JavaScript.

The final application contains ten Software Engineering study documents covering testing, requirements engineering, database normalization, software development processes, UML, Git, object-oriented programming, software security, algorithms, and project management. These documents are divided into searchable chunks and stored locally. When a user submits a question, the system retrieves the most relevant chunks, adds them to the model prompt, and generates a grounded response with source attribution.

Additional improvements include a responsive interface, compact answer mode, three visual themes, recent-conversation storage, collapsible study topics, runtime document uploads, stop-word filtering, unsupported-question protection, and mobile support. The completed system passed all 51 automated tests.

1. Introduction

Artificial intelligence assistants are increasingly used to answer questions and support learning. However, a standard language model may provide inaccurate or unsupported information because it normally answers using general knowledge learned during training. It may also produce an answer even when it does not have reliable information.

Retrieval-Augmented Generation addresses this problem by combining document retrieval with language-model generation. In a RAG system, relevant sections are first retrieved from a collection of documents. These sections are then supplied to the language model as context, allowing it to generate an answer based on the provided material rather than only its general training data. The basic process consists of retrieving relevant chunks, augmenting the model prompt with those chunks, and generating a response grounded in the documents.

The objective of this project was to adapt a local RAG starter application into a Software Engineering Study Assistant. The final application runs on the user's computer and does not depend on a cloud AI service or online API key after the model has been downloaded.

The system was designed to help students study Software Engineering topics by asking natural-language questions. It retrieves relevant content from local study files and displays concise answers together with the sources used.

2. Problem Definition

Software Engineering students study many different subjects, including software testing, databases, requirements, system design, object-oriented programming, security, algorithms, project management, and version control.

Course information is often divided between lecture notes, documents, slides, and personal study material. Finding a particular definition or comparison may therefore take time. A traditional keyword search can locate individual words, but it does not provide a clear explanation in natural language.

A general-purpose AI assistant may answer quickly, but it may also:

- Use information outside the course material
- Produce incorrect or invented details
- Give answers that do not match the lecturer's content
- Fail to show where its information came from
- Require an internet connection or paid API access

The proposed solution is a local study assistant that searches a defined Software Engineering knowledge base and generates answers grounded in that material.

3. Project Objectives

The main objective of the project was to build a functional local RAG application for Software Engineering education.

The specific objectives were:

1. To run the language model locally on the computer.
2. To avoid dependence on cloud AI APIs.
3. To prepare a structured Software Engineering knowledge base.
4. To divide documents into searchable chunks.
5. To retrieve the most relevant chunks for each question.
6. To generate answers based only on the retrieved content.
7. To display document sources with each supported answer.
8. To reject questions that cannot be answered from the local knowledge base.
9. To support the addition of new Markdown and text documents.
10. To provide a modern and responsive user interface.
11. To support both detailed and compact answers.
12. To save recent conversations in the browser.
13. To test the application using automated and manual tests.

4. Background

4.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation is an AI application pattern that connects a language model to an external knowledge source.

Instead of giving the model every document, the system searches for the most relevant document sections. Only these sections are inserted into the prompt. This makes the process more token-efficient and allows the model to provide answers that are traceable to specific sources.

The main RAG stages are:

1. Document ingestion
2. Document chunking
3. Vector or term representation
4. Storage
5. Query processing
6. Similarity search
7. Context construction
8. Answer generation
9. Source presentation

RAG is especially useful when a system must work with private, domain-specific, or frequently updated information.

4.2 Foundry Local

Foundry Local is Microsoft's on-device AI runtime. It downloads, manages, and loads compatible language models on the local computer. According to the project guide, it supports in-process inference through a native SDK, automatic model management, and hardware-optimized model selection.

This project uses Foundry Local with Phi-3.5 Mini. Once the model has been downloaded, the assistant can operate without sending prompts or documents to an external AI API.

4.3 TF-IDF

TF-IDF stands for Term Frequency–Inverse Document Frequency. It is a numerical method used to represent the importance of words inside documents.

Term Frequency measures how often a term appears in a document or chunk. Inverse Document Frequency reduces the importance of terms that appear in many documents.

This project uses TF-IDF because it is:

- Fully local
- Fast
- Transparent
- Suitable for a small domain-specific knowledge base
- Independent of an additional embedding model

The reference application also uses TF-IDF with SQLite because it provides simple and fast retrieval for a small document collection.

4.4 Cosine Similarity

Cosine similarity compares the direction of two term vectors. In this application, it measures the similarity between the user's question and each stored document chunk.

A higher score indicates that the question and chunk share more relevant terms. The chunks are ranked by their similarity score, and the highest-ranked results are sent to the language model.

4.5 Document Chunking

Long documents cannot always be inserted into a prompt in full. Therefore, every document is divided into smaller overlapping chunks.

The overlap helps preserve information that might otherwise be divided across two chunks. The reference RAG pipeline uses approximately 200-token chunks with overlap and retrieves the most relevant chunks during a query.

5. Technologies Used

The project uses the following technologies:

Technology	**Purpose**
Node.js	JavaScript runtime for the server
Express.js	HTTP server and API endpoints

Foundry Local	Local AI runtime
Phi-3.5 Mini	Local language model
SQLite	Persistent document and chunk storage
better-sqlite3	Node.js interface for SQLite
TF-IDF	Keyword-based document representation
Cosine similarity	Query and chunk similarity calculation
HTML	User-interface structure
CSS	Themes, layout, and responsiveness
JavaScript	Client-side chat and interface logic
Server-Sent Events	Streaming model responses
Git	Version control
Node.js Test Runner	Automated tests

The starter architecture intentionally uses a simple stack: Node.js and Express for the server, SQLite for storage, TF-IDF with cosine similarity for retrieval, and a single HTML interface.

6. System Architecture

The system contains five main layers.

6.1 Client Layer

The client layer is the web interface displayed in the browser.

It includes:

- Chat messages
- Question input
- Send button
- Study topic navigation
- Recent conversations

- Source cards
- Compact answer mode
- Theme selection
- Knowledge-base upload panel
- Mobile sidebar

6.2 Server Layer

The Express server manages communication between the browser and the application logic.

It provides endpoints for:

- Application health
- Model status
- Standard chat responses
- Streaming chat responses
- Document listing
- Runtime document upload

6.3 RAG Pipeline

The RAG pipeline is responsible for:

- Converting the question into a term-frequency vector
- Searching the document store
- Ranking chunks
- Constructing the model prompt
- Supplying context to the local model
- Returning generated text and source information

6.4 Data Layer

The data layer contains:

- Markdown study documents
- Uploaded text documents

- SQLite database
- Document metadata
- Document chunks
- TF-IDF information

6.5 AI Layer

The AI layer uses:

- Foundry Local
- Foundry Local JavaScript SDK
- Phi-3.5 Mini
- Streaming chat completion

The reference architecture similarly divides the application into client, server, RAG pipeline, data, and AI layers, all running on one machine.

7. System Workflow

The complete workflow is:

Software Engineering documents
↓
Document parsing
↓
Overlapping chunk generation
↓
TF-IDF term vectors
↓
SQLite storage
↓
User question
↓
Stop-word filtering
↓
Question term vector
↓

Cosine-similarity search
↓
Top relevant chunks
↓
Prompt construction
↓
Phi-3.5 Mini through Foundry Local
↓
Generated answer
↓
Answer and retrieved sources displayed

During ingestion, every Markdown file is read and parsed. Optional metadata such as title, category, and document ID is extracted from YAML front matter.

The document is then divided into chunks. Each chunk is stored in SQLite with its document metadata and term-frequency representation.

When a user submits a question, common stop words are removed. The remaining terms are compared with the indexed chunks. If relevant chunks are found, they are inserted into the system prompt. The local language model then generates a response.

If no relevant chunks are found, the model is not called. Instead, the application returns:

This information is not available in the local knowledge base.

This deterministic check prevents the model from answering unsupported questions using general knowledge.

8. Knowledge-Base Development

The original gas-field knowledge base was replaced with ten Software Engineering documents.

8.1 Included Topics

The final knowledge base contains:

1. Software Testing Fundamentals
2. Requirements Engineering Fundamentals
3. Database Design and Normalization
4. Software Development Life Cycle and Agile Methods

5. Software Architecture and UML Fundamentals
6. Git and Version Control Fundamentals
7. Object-Oriented Programming Fundamentals
8. Software Security Fundamentals
9. Algorithms and Data Structures Fundamentals
10. Software Project Management Fundamentals

8.2 Document Format

Each document uses Markdown and optional YAML front matter.

Example:

```
---
title: Software Testing Fundamentals
category: Software Testing
id: SE-TEST-001
---

# Software Testing Fundamentals

Software testing is the process of evaluating a software system...
```

The metadata allows the application to display clear document titles and categories in the source cards.

8.3 Ingestion Result

The final ingestion process produced:

- 10 documents
- 42 searchable chunks
- One local SQLite database

The database can be rebuilt at any time using:

```
npm run ingest
```

```
ceren@ceren-ASUS-TUF-Gaming-F17-FX707ZC4-FX707ZC4:~/study-assistant-clean-test$ npm run ingest

> software-engineering-study-assistant@2.0.0 ingest
> node src/ingest.js

=== Software Engineering Study Assistant — Document Ingestion ===

Found 10 documents.

  ✓ 01-software-testing.md → 2 chunk(s)  [Software Testing]
  ✓ 02-requirements-engineering.md → 3 chunk(s)  [Requirements Engineering]
  ✓ 03-database-design.md → 4 chunk(s)  [Database Systems]
  ✓ 04-sdlc-and-agile.md → 4 chunk(s)  [Software Process]
  ✓ 05-software-architecture-and-uml.md → 5 chunk(s)  [Software Design]
  ✓ 06-git-and-version-control.md → 3 chunk(s)  [Software Development Tools]
  ✓ 07-object-oriented-programming.md → 4 chunk(s)  [Software Design]
  ✓ 08-software-security.md → 4 chunk(s)  [Software Security]
  ✓ 09-algorithms-and-data-structures.md → 7 chunk(s)  [Computer Science]
  ✓ 10-software-project-management.md → 6 chunk(s)  [Project Management]

Ingestion complete: 42 chunks from 10 documents.
Database: /home/ceren/study-assistant-clean-test/data/rag.db
```

Figure 1. Successful ingestion of ten Software Engineering documents.

9. Implementation

9.1 Initial Project Setup

The project was cloned from the local RAG starter repository. Dependencies were installed using:

npm install

The original project used an older Foundry Local SDK version. It initially produced an SSL and model-catalog error. The SDK was updated from version 0.9.0 to version 1.2.3.

After the update, the model downloaded and loaded successfully.

9.2 Streaming Compatibility Update

The original starter code used callback-based streaming. The updated SDK returns an asynchronous iterable.

The streaming code was changed to use:

```
for await (const chunk of this.chatClient.completeStreamingChat(messages)) {
   // Read and display streamed content
}
```

This change resolved the error:

Tools must be an array if provided.

9.3 System Prompt

The gas-field safety prompt was replaced with a Software Engineering study prompt.

The new prompt instructs the model to:

- Answer only from retrieved context
- Avoid outside knowledge
- Avoid invented information
- Produce concise answers
- Avoid unnecessary introductions
- Use short paragraphs or bullet points
- Return a fixed refusal when information is unavailable

Two prompt modes are supported:

- Normal mode
- Compact mode

9.4 Stop-Word Filtering

During early testing, the assistant incorrectly answered the question:

What is the capital of France?

Unrelated documents were retrieved because common terms such as “what,” “is,” “the,” and “of” appeared throughout the knowledge base.

A stop-word list was added to the tokenizer. Common English words are removed before similarity calculations.

After this improvement, the France question produced no relevant chunks.

9.5 Unsupported-Question Protection

A second protection was added to the chat engine.

If retrieval returns no chunks, the application does not send the question to the language model. It immediately returns the fixed refusal message.

This prevents the model from using its pre-trained general knowledge.

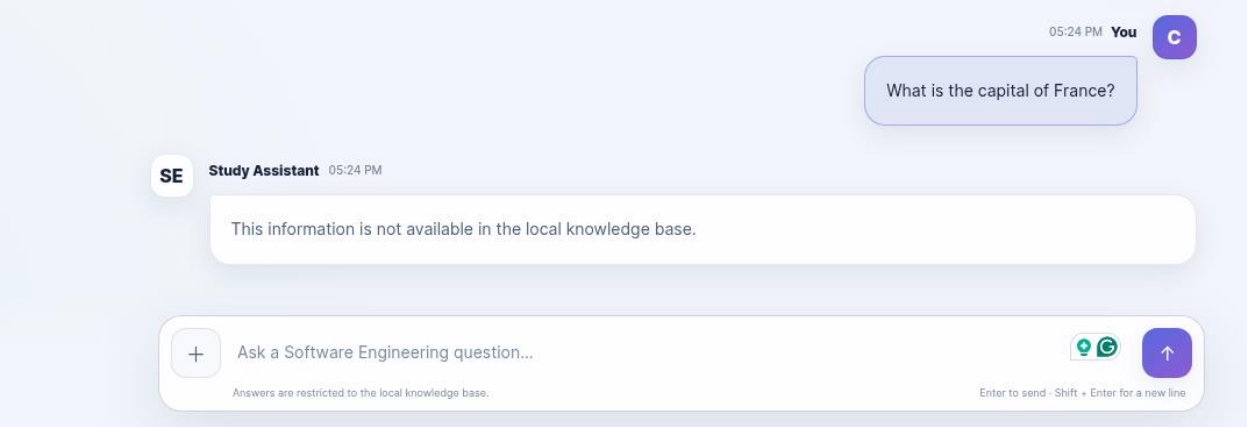

Figure 2. Unsupported question rejected because the answer was not present in the local knowledge base.

9.6 Runtime Document Upload

The interface allows the user to upload .md and .txt files.

An uploaded document is:

1. Validated
2. Parsed
3. Divided into chunks
4. Vectorized
5. Added to SQLite
6. Made searchable immediately

The application supports a maximum upload size of 2 MB.

The reference application also supports adding documents at runtime without restarting the server.

A DevOps test document was uploaded during testing. The assistant successfully answered a question about Continuous Integration and showed the uploaded document as the strongest source.

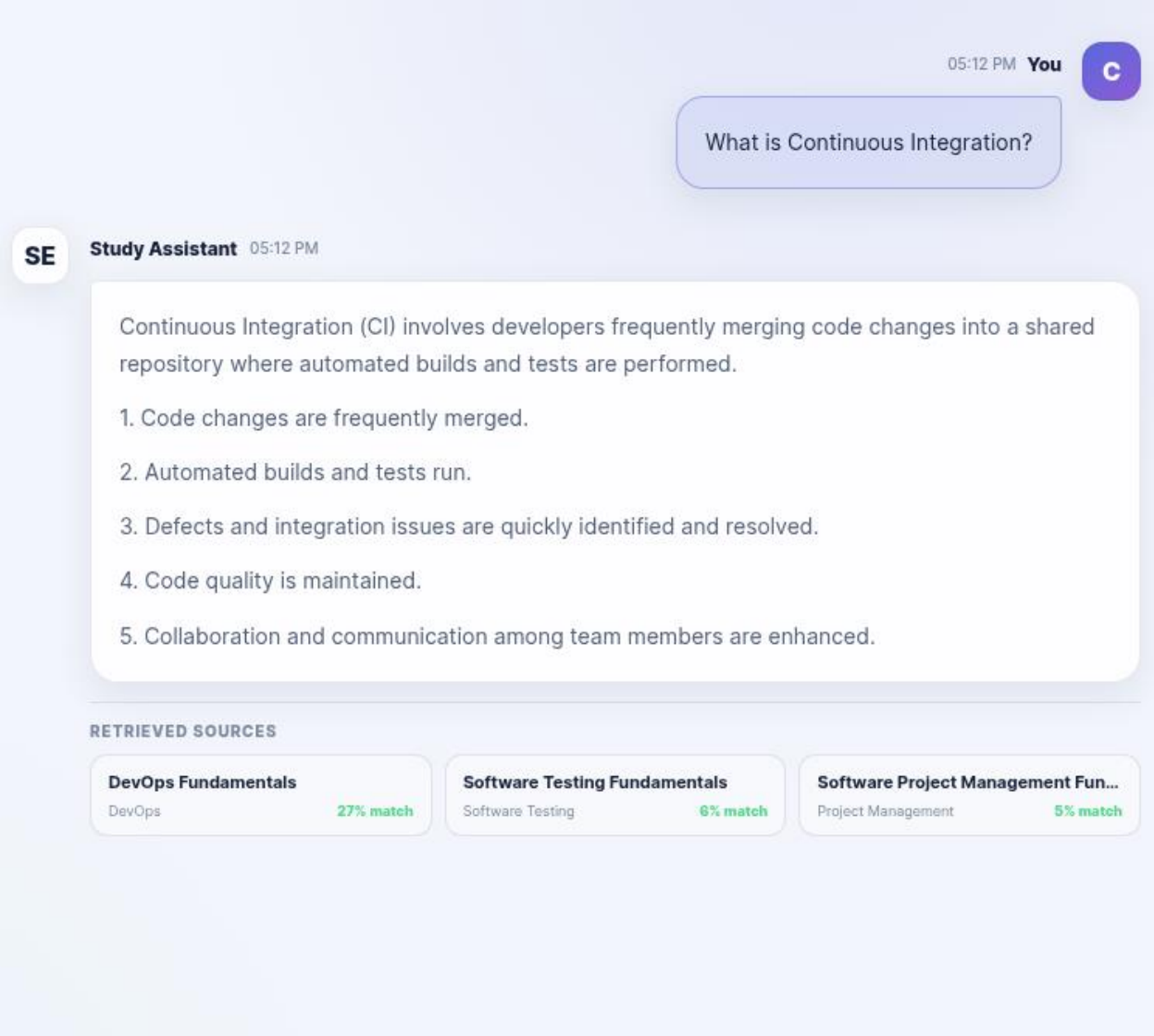

Figure 3. Question answered using a document uploaded at runtime.

9.7 Recent Conversations

Recent conversations are stored in browser localStorage.

The sidebar shows the most recent questions. Selecting a recent item restores:

- The user question
- The assistant answer
- The source cards
- The original conversation layout

A clear button allows all recent items to be removed.

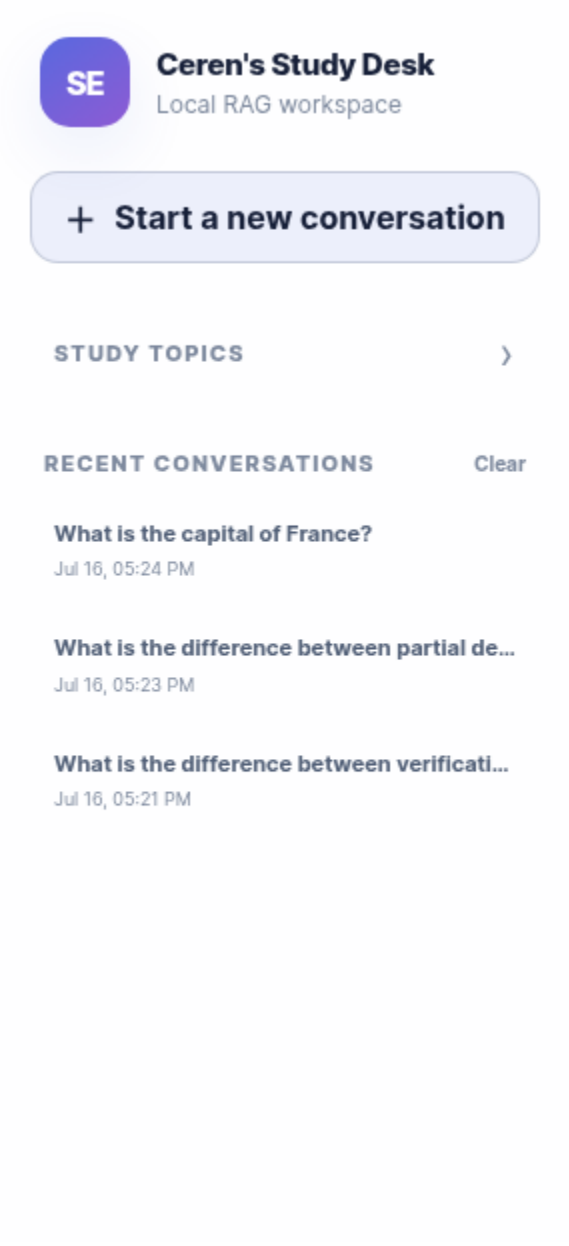

Figure 4. Recent conversations saved in the sidebar.

10. User-Interface Design

The original industrial gas-field interface was replaced completely.

The redesigned interface uses a study-oriented workspace containing:

- Sidebar navigation
- Main chat area
- Suggested questions
- Source cards
- Fixed input composer
- Model status
- Knowledge-base panel
- Mobile navigation
- User and assistant avatars

10.1 Message Alignment

User messages appear on the right side.

The user header is displayed in this order:

Time → You → C

Assistant messages remain on the left with the “SE” avatar.

10.2 Collapsible Study Topics

The ten topic buttons are placed inside a collapsible Study Topics section. This reduces sidebar space usage.

The section remains open after a topic is selected and closes only when the user manually clicks the section heading.

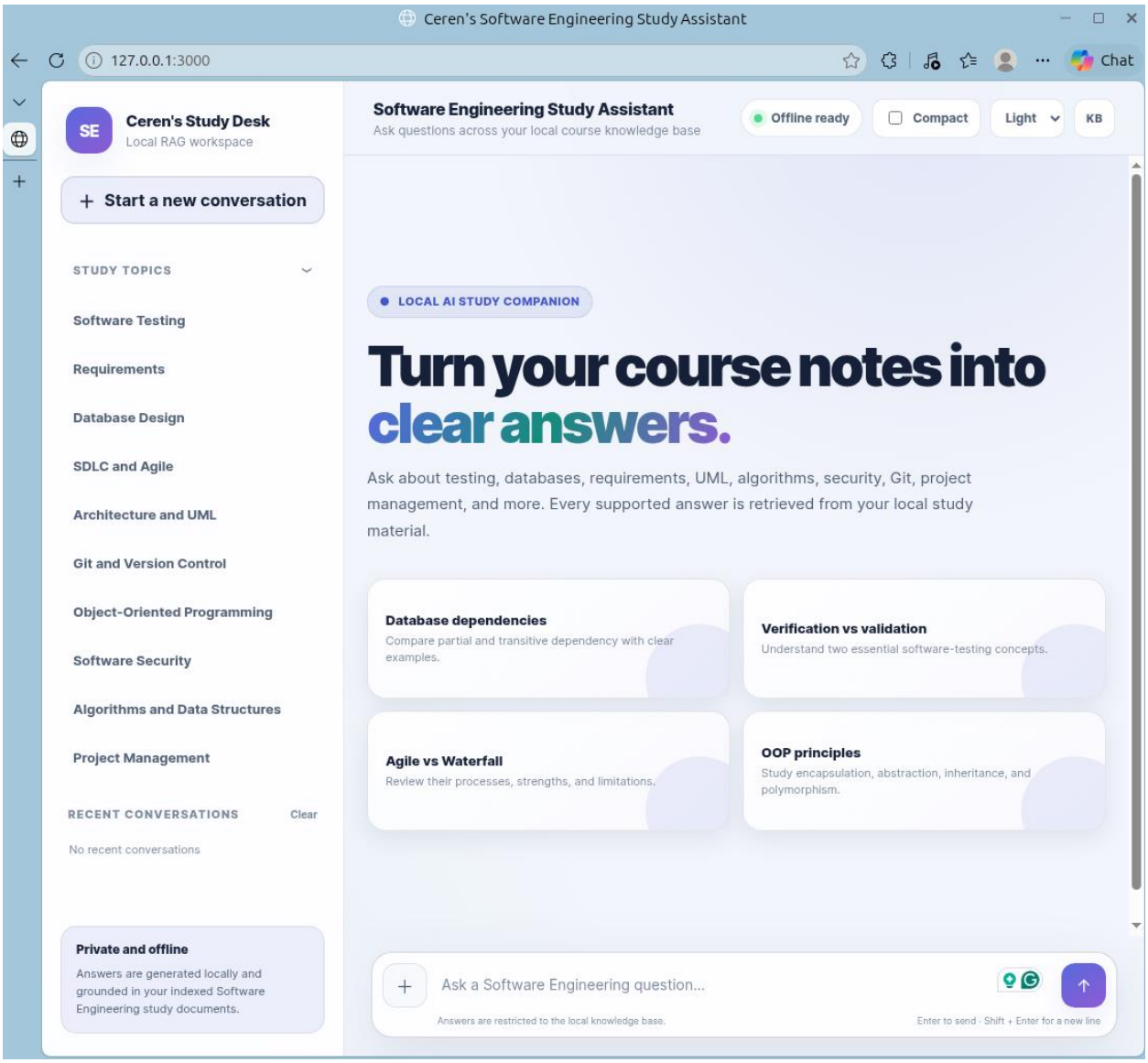

Figure 5. Collapsible Study Topics navigation.

10.3 Themes

The application contains three visual themes:

- Dark
- Light
- Rose

The selected theme is saved locally and restored when the page is refreshed.

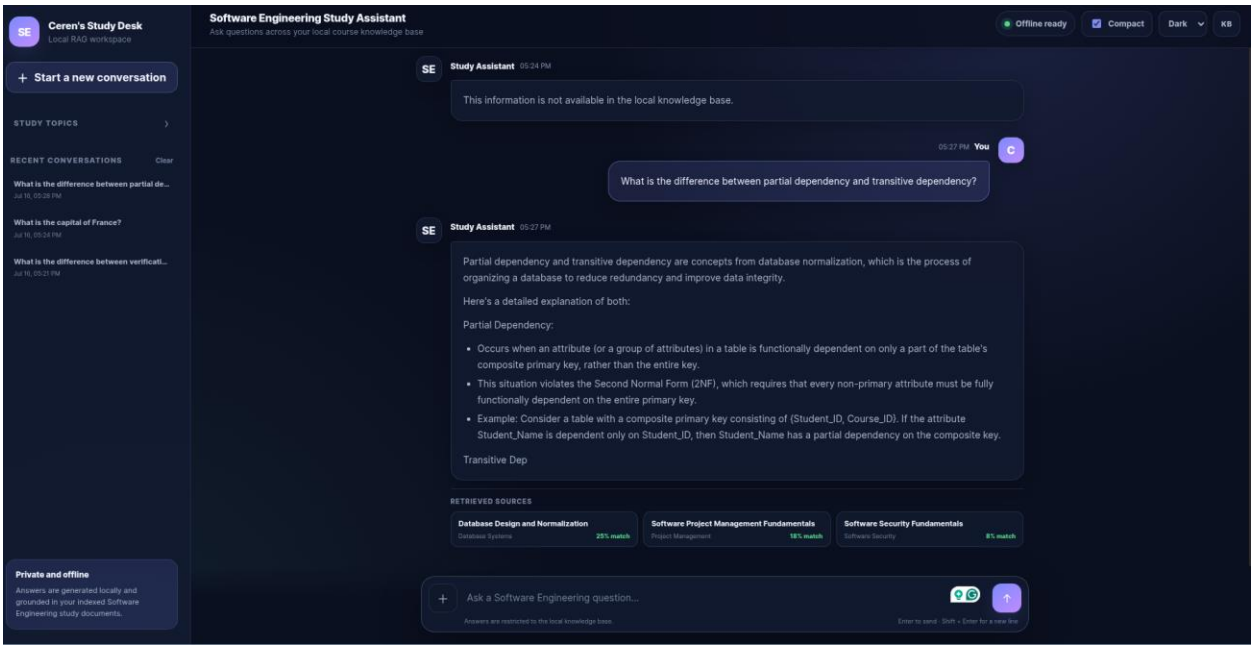

Figure 6. Alternative theme view 1.

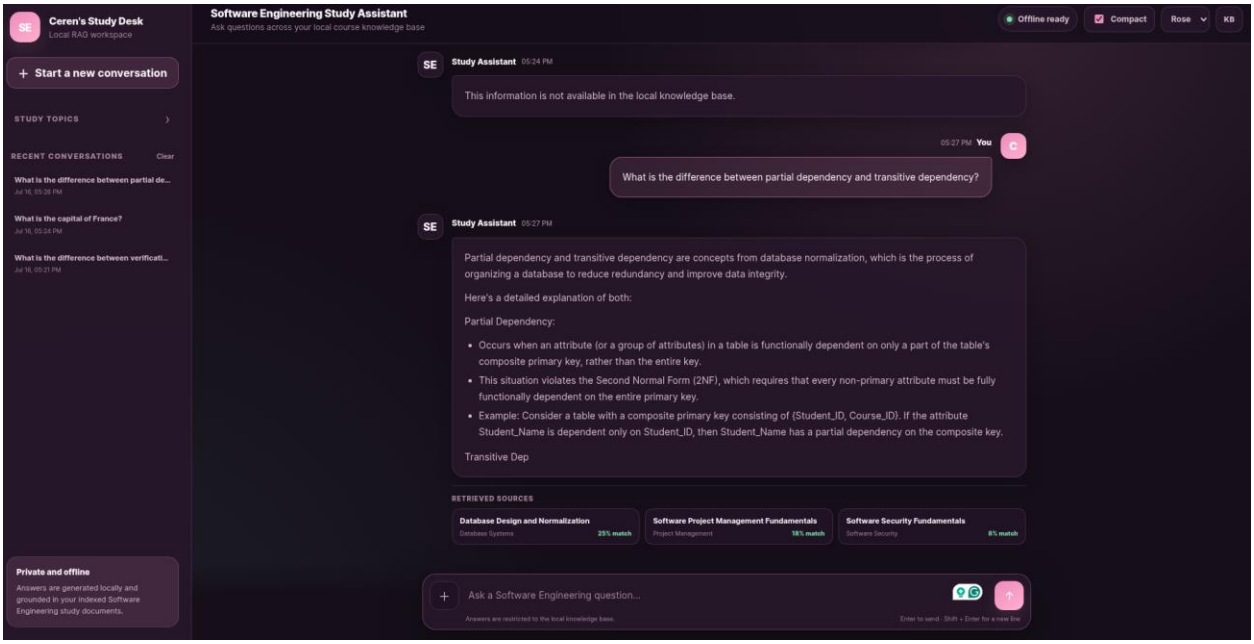

Figure 7. Alternative theme view 2.

10.4 Responsive Design

The interface was tested at desktop, tablet, and mobile widths.

Mobile improvements include:

- Hidden desktop-only controls
- Mobile sidebar
- Smaller typography
- Single-column suggestion cards
- Full-width message layout
- Responsive composer
- Horizontal-overflow prevention
- Source cards displayed vertically

The reference application also emphasizes a mobile-responsive design and support for screen widths beginning at approximately 320 pixels.

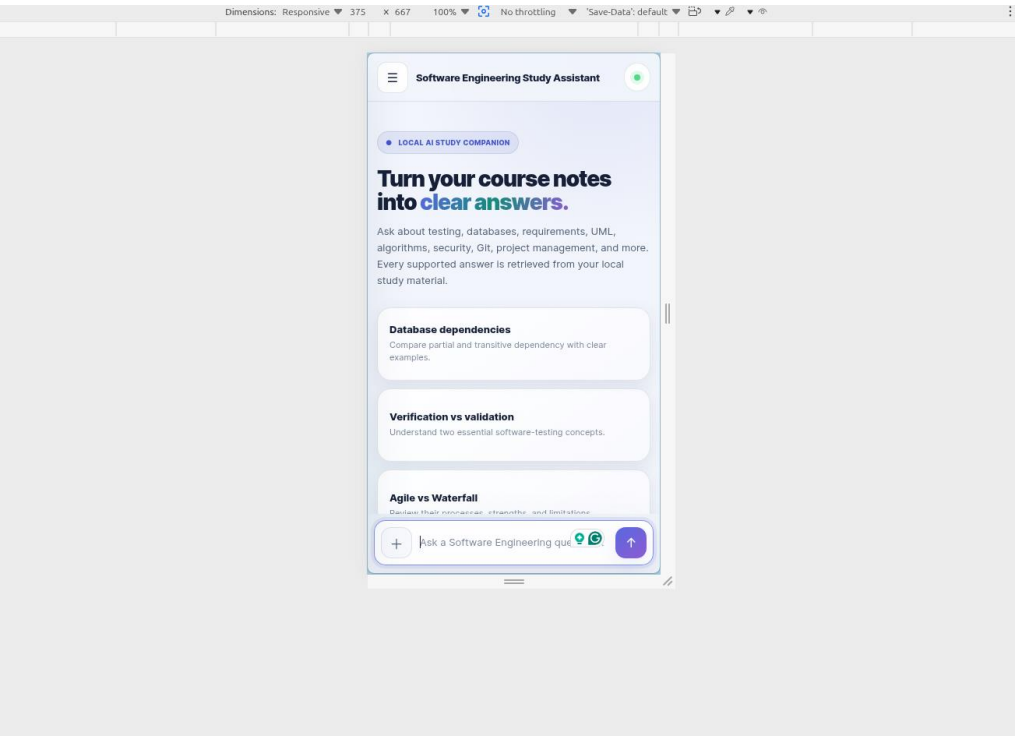

Figure 8. First mobile interface at approximately 375 × 667 pixels.

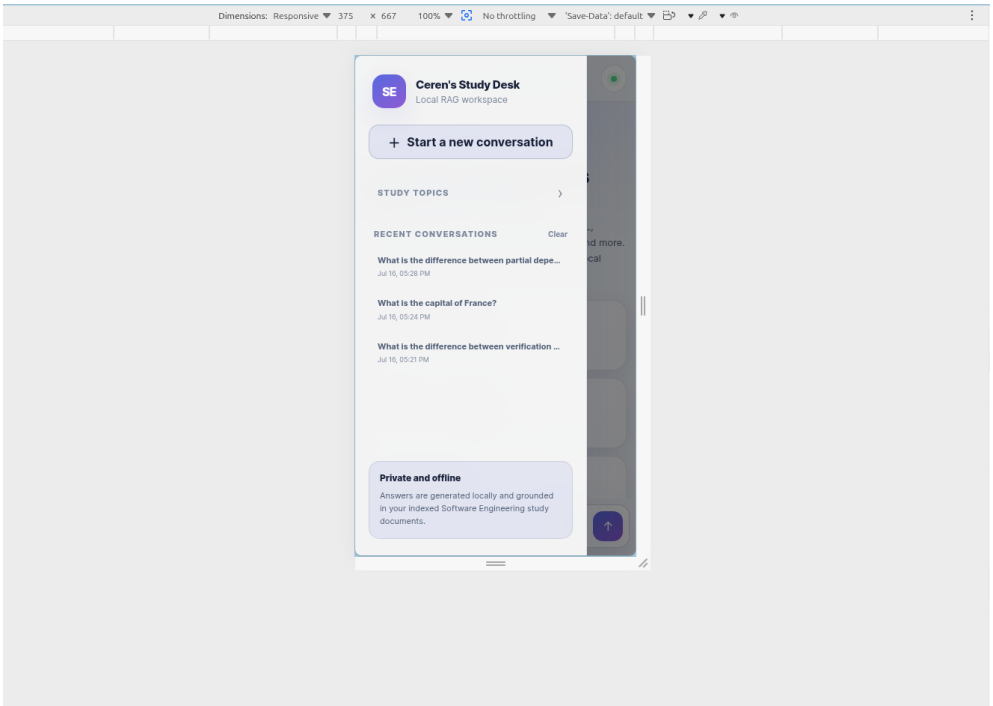

Figure 9. Second mobile interface at approximately 375 × 667 pixels.

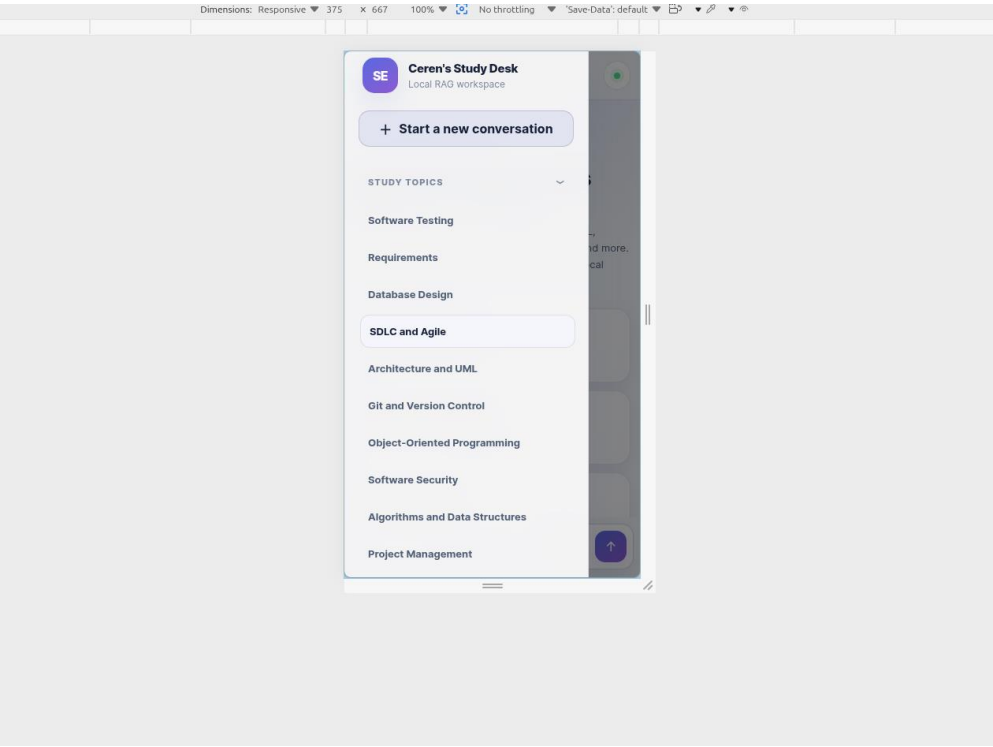

Figure 10. Third mobile interface at approximately 375 × 667 pixels.

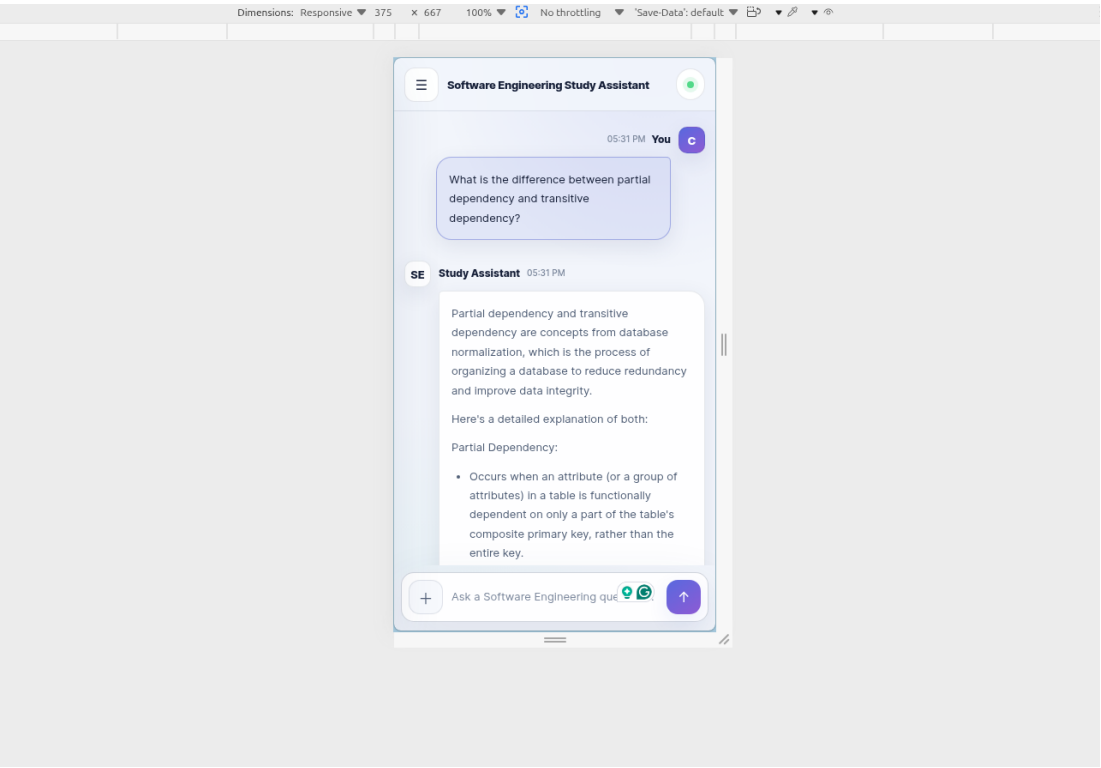

Figure 10. Fourth mobile interface at approximately 375 × 667 pixels.

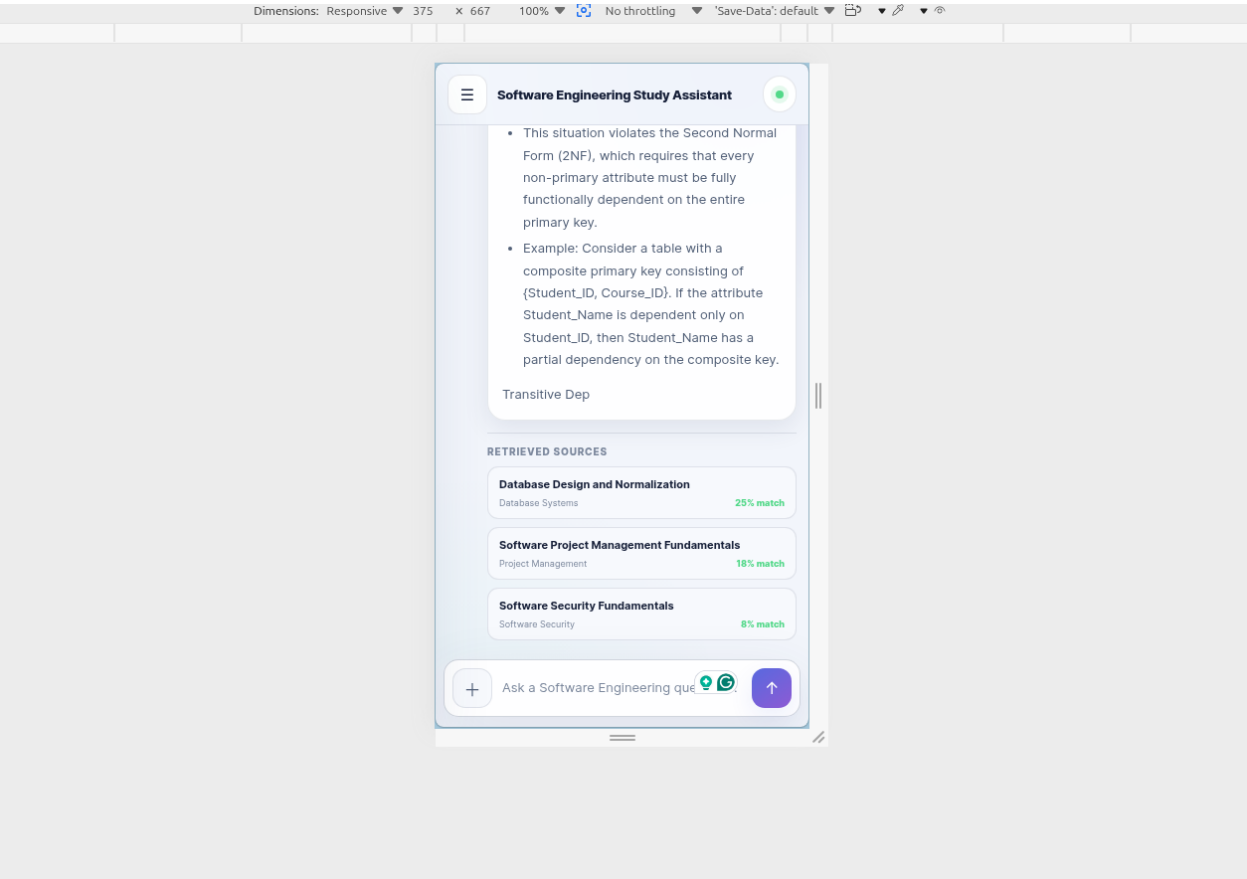

Figure 11. Fifth mobile interface at approximately 375 × 667 pixels.

11. Testing

Testing was performed using both automated and manual methods.

11.1 Automated Testing

The project uses the built-in Node.js test runner.

The command used was:

npm test

The final result was:

Tests: 51
Passed: 51
Failed: 0

The automated tests cover:

- YAML front-matter parsing
- Document chunking
- Chunk overlap
- Empty document handling
- Term-frequency counting
- Stop-word filtering
- Cosine similarity
- Application configuration
- System prompts
- Health endpoint
- Document listing
- Chat endpoint validation
- Runtime uploads
- Filename sanitization
- SQLite insertion
- SQLite retrieval
- Document removal
- Database clearing

The original project also includes unit tests for the chunker, vector store, configuration, and server endpoints.

Figure 12. All 51 automated tests passing.

11.2 Manual Test Results

Test	Input or Action	Expected Result	Actual Result	Status
1	Verification vs validation	Correct testing comparison	Correct answer with testing source	Pass
2	Partial vs transitive dependency	Correct database explanation	Correct answer with database source	Pass
3	Agile vs Waterfall	Clear comparison	Correct concise comparison	Pass
4	SQL injection	Definition and prevention methods	Correct security answer	Pass
5	BFS vs DFS	Correct algorithm comparison	Correct answer with algorithm source	Pass
6	Capital of France	Knowledge-base refusal	Fixed refusal with no sources	Pass
7	Empty input	No message submitted	No request sent	Pass
8	Compact mode	Shorter answer	Answer length reduced	Pass
9	Runtime upload	New file becomes searchable	Uploaded DevOps file retrieved	Pass
10	Recent conversations	Conversation persists after refresh	Conversation restored correctly	Pass
11	New conversation	Current chat cleared	Welcome screen restored	Pass
12	Mobile width	No horizontal overflow	Interface displayed correctly	Pass
13	Theme change	Theme remains readable	All three themes worked	Pass
14	Clean installation	Project runs from fresh clone	Installation and ingestion succeeded	Pass

11.3 Clean-Installation Test

A fresh clone was created in a separate directory.

The following commands were executed:

npm install
npm run ingest
npm start

The knowledge base was rebuilt successfully, and the application answered both supported and unsupported questions correctly.

This confirmed that the committed project contains everything required to install and run the application.

12. Results

The final application successfully achieved the main objectives.

The assistant:

- Runs locally
- Loads Phi-3.5 Mini through Foundry Local
- Indexes Software Engineering documents
- Retrieves relevant chunks
- Produces grounded answers
- Displays document sources
- Refuses unsupported questions
- Supports runtime uploads
- Streams answers in real time
- Stores recent conversations
- Works on desktop and mobile
- Passes all automated tests

A supported question such as:

What is the difference between verification and validation?

produces an answer grounded in the Software Testing Fundamentals document.

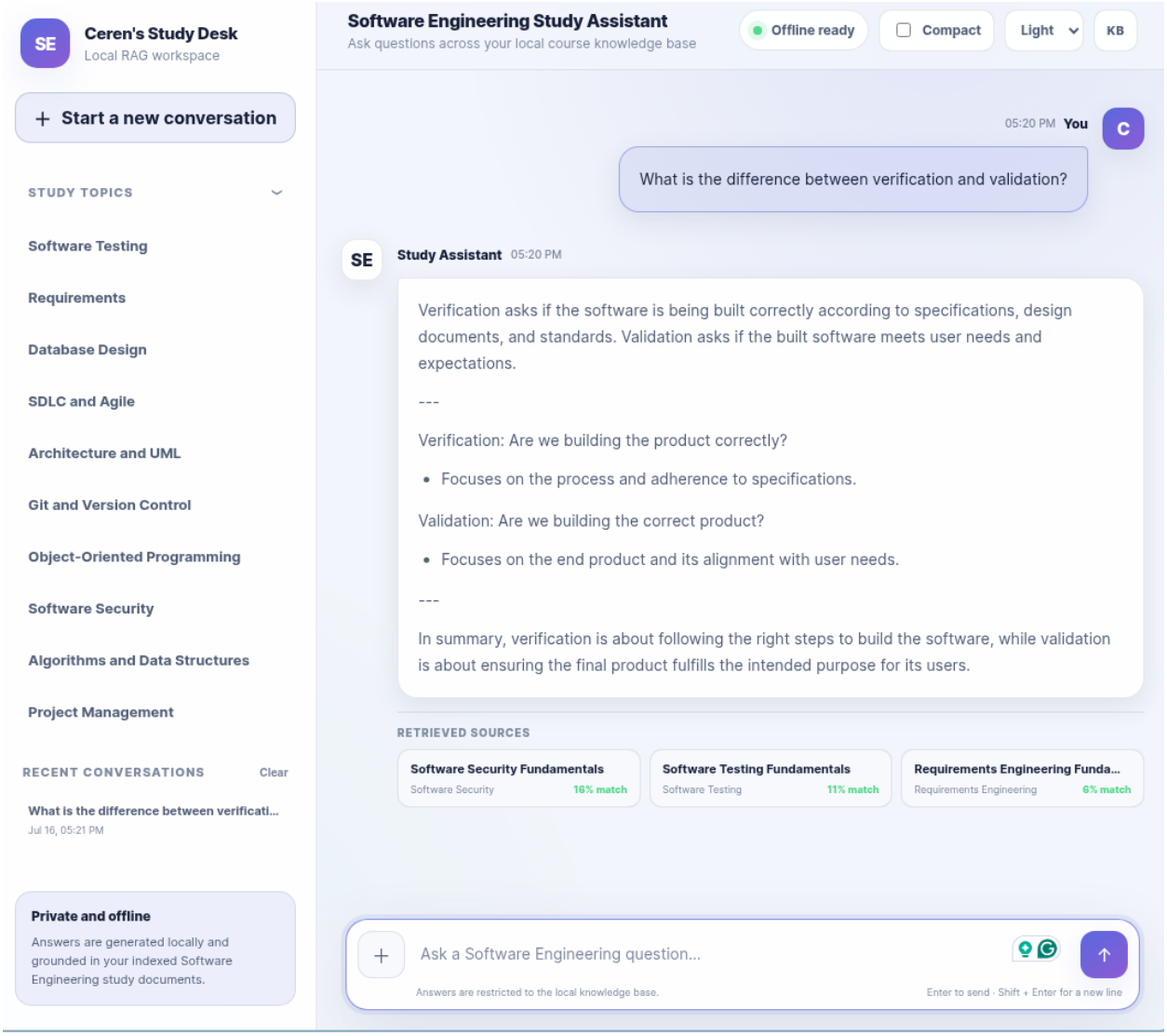

Figure 13. Correct answer with retrieved source attribution.

A database question correctly retrieves the Database Design and Normalization document.

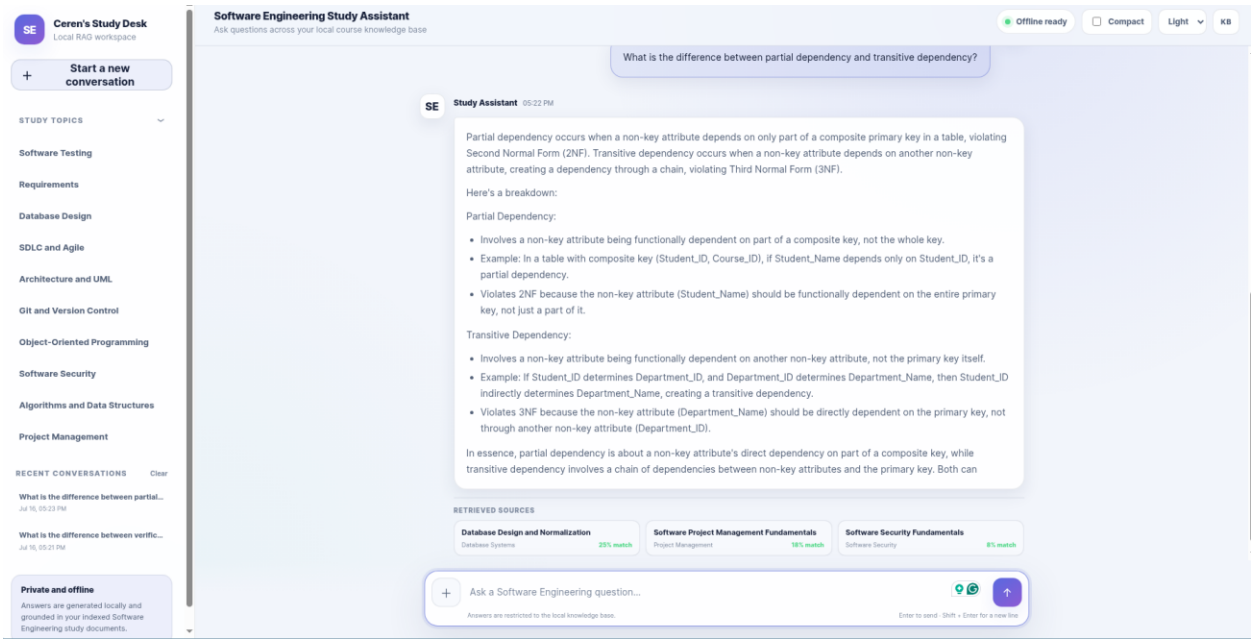

Figure 14. Database question answered using the relevant knowledge-base document.

The final server output confirms that the model and vector store load successfully.

```
ceren@ceren-ASUS-TUF-Gaming-F17-FX707ZC4-FX707ZC4:~/local-rag$ npm start

> software-engineering-study-assistant@2.0.0 start
> node src/server.js

=== Software Engineering Study Assistant ===

[ChatEngine] Initializing Foundry Local SDK...
[ChatEngine] Discovering available models...
[Server] UI available at http://127.0.0.1:3000
[Server] Initializing model in background...

[ChatEngine] Selected model: phi-3.5-mini
[ChatEngine] Model phi-3.5-mini is already cached.
[ChatEngine] Loading phi-3.5-mini into memory...
[ChatEngine] Model ready: phi-3.5-mini
[ChatEngine] Vector store ready: 43 chunks indexed.
[Server] Fully offline — no outbound connections.
```

Figure 15. Local model and vector store ready.

13. Improvements Made to the Starter Project

The following major changes were completed:

1. Replaced the gas-field knowledge base with Software Engineering documents.
2. Renamed the project internally and visually.
3. Updated the Foundry Local SDK.
4. Updated callback streaming to asynchronous streaming.
5. Rewrote the system prompts.
6. Added concise answer limits.
7. Added a compact answer mode.
8. Added stop-word filtering.
9. Added deterministic unsupported-question protection.
10. Redesigned the full user interface.
11. Added right-aligned user messages.
12. Added responsive mobile styling.
13. Added dark, light, and rose themes.
14. Added collapsible Study Topics.
15. Added recent-conversation storage.
16. Added source cards.
17. Added runtime document uploads.
18. Updated the README.
19. Updated automated tests for the study domain.
20. Verified the project through a clean-install test.

14. Limitations

Although the application works successfully, it has several limitations.

14.1 Keyword-Based Retrieval

TF-IDF mainly relies on shared vocabulary. A question may fail to retrieve a relevant document if it uses completely different wording from the document.

14.2 Small Local Model

Phi-3.5 Mini is efficient and suitable for local use, but its answer quality may be lower than that of much larger cloud models.

14.3 Knowledge-Base Dependency

The assistant can answer only using the available documents. Missing, incomplete, or inaccurate notes directly affect answer quality.

14.4 Limited File Formats

Runtime upload currently supports only:

- Markdown
- Plain text

PDF, Word, PowerPoint, and image files are not yet supported.

14.5 Browser-Based Conversation Storage

Recent conversations are stored using localStorage. They are available only in the same browser and device.

14.6 No User Authentication

The current application is designed for one local user and does not include login or role management.

14.7 Basic Source Scoring

The interface may display up to three retrieved sources, including low-scoring secondary matches. A future version could apply a minimum relevance threshold before displaying sources.

15. Future Improvements

The project could be improved in the following ways:

15.1 Local Embedding Model

A local embedding model could improve semantic retrieval and reduce dependence on exact keyword overlap.

15.2 Hybrid Retrieval

TF-IDF and semantic embeddings could be combined to provide both keyword precision and semantic understanding.

15.3 PDF and Word Support

A document-processing layer could extract text from:

- PDF files
- Word documents
- PowerPoint slides

15.4 Quiz Mode

The assistant could automatically generate:

- Multiple-choice questions
- Short-answer questions
- Practice exams
- Self-assessment exercises

15.5 Flashcards

Retrieved course content could be transformed into interactive flashcards.

15.6 Persistent Database Conversations

Conversation history could be stored in SQLite rather than only in browser storage.

15.7 Conversation Export

Users could export conversations as:

- Text
- Markdown

- PDF

15.8 Document Management

The interface could allow users to:

- Delete documents
- Rename documents
- Edit metadata
- Re-index individual files

15.9 Minimum Source Threshold

Low-relevance sources could be hidden by requiring a minimum similarity score.

15.10 Progressive Web Application

The application could be packaged as an installable Progressive Web App for a more application-like experience.

16. Conclusion

This project successfully transformed a local gas-field RAG starter application into a Software Engineering Study Assistant.

The completed application runs locally and uses Foundry Local with Phi-3.5 Mini. It retrieves relevant sections from ten Software Engineering documents using TF-IDF and cosine similarity. The retrieved chunks are supplied to the language model, which generates concise answers grounded in the local knowledge base.

The system also provides source attribution, runtime document upload, recent conversations, compact responses, responsive layouts, multiple themes, and unsupported-question protection.

One of the most important improvements was preventing the model from answering questions outside the knowledge base. Stop-word filtering and a hard no-context rule were added after testing revealed that common words could cause unrelated chunks to be retrieved.

The final system passed all 51 automated tests and also passed manual and clean-install testing. Therefore, the project objectives were achieved.

The application demonstrates that a useful domain-specific AI assistant can be developed without a cloud AI budget or continuous internet connection. Foundry Local, Node.js, SQLite, TF-IDF, and a compact local language model were sufficient to create a private and functional educational RAG system.

References

Stott, L. (2026). *Building Your First Local RAG Application with Foundry Local*. Microsoft Developer Community Blog, Microsoft.

Microsoft. *Foundry Local SDK and on-device model runtime*.

Node.js. *Node.js JavaScript Runtime Documentation*.

Express.js. *Express Web Framework Documentation*.

SQLite. *SQLite Database Documentation*.

Appendix A — Application Commands

Install dependencies

npm install

Build the local knowledge base

npm run ingest

Start the application

npm start

Run automated tests

npm test

Application address

http://127.0.0.1:3000

Appendix B — Suggested Screenshot Placement

Figure	Screenshot filename	Description
Figure 1	12-ingestion-success.png	Successful ingestion of the Software Engineering knowledge-base documents
Figure 2	05-unsupported-question.png	Unsupported question rejected because the answer was not available in the local knowledge base
Figure 3	07-runtime-document-upload.png	Continuous Integration question answered using the uploaded DevOps document
Figure 4	06-recent-conversations.png	Recent conversations displayed in the sidebar
Figure 5	02-study-topics-expanded.png	Collapsible Study Topics section expanded
Figure 6	10.1-theme-view.png	Dark-theme interface

Figure 7	10.2-theme-view.png	Rose-theme interface
Figure 8	11.1-mobile-view.png	Mobile welcome interface at approximately 375 × 667 pixels
Figure 9	11.2- mobile-view.png	Mobile sidebar showing recent conversations
Figure 10	11.3- mobile-view.png	Mobile sidebar showing the expanded Study Topics section
Figure 11	11.4- mobile-view.png	Mobile view of a user question and assistant response
Figure 12	11.5- mobile-view.png	Mobile response showing retrieved source cards
Figure 13	13-tests-passed.png	Terminal output showing all 51 automated tests passing
Figure 14	03-correct-answer-with-sources.png	Verification and validation answer with retrieved source attribution
Figure 15	04-database-question.png	Partial and transitive dependency answer using the database knowledge-base document
Figure 16	14-server-ready.png	Terminal output showing the local model and vector store ready